

知识库系统部署与运维手册

一、环境要求

生产环境的最低硬件配置：CPU 不少于 8 核（推荐 16 核以上），内存不少于 32GB（推荐 64GB 以上），可用磁盘空间不少于 500GB SSD（推荐 NVMe SSD），网络带宽不少于 100Mbps。软件环境要求：操作系统为 Ubuntu 22.04 LTS 或 Rocky Linux 9.x；容器运行时为 Docker Engine 24.x+ 及 Docker Compose v2.x+；Python 版本为 3.12+（仅在本地开发部署时需要）；数据库外部依赖包括 PostgreSQL 16+、Redis 7+ 和 Neo4j 5.x Community Edition。

二、Docker Compose 部署

2.1 服务拓扑

项目根目录下的 docker-compose.yml 文件定义了完整的生产级服务拓扑，包含 8 个核心服务组件：api 服务负责接收和处理所有 HTTP 请求，对外暴露 8000 端口；worker 服务基于 Celery 实现异步任务队列，专门处理文档解析等耗时操作，配置了 4 个并发 Worker 进程；milvus-standalone 服务使用 Milvus 2.4.0 镜像作为分布式向量数据库，对外暴露 19530 (gRPC) 和 9091 (Metrics) 端口；etcd 服务作为 Milvus 的元数据协调中心，使用 3.5.5 版本；minio 服务提供 S3 兼容的对象存储，同时为 Milvus 和文档存储提供后端，对外暴露 9000 (API) 和 9001 (Console) 端口；neo4j 服务使用 5.x Community 版本作为知识图谱数据库，对外暴露 7474 (HTTP) 和 7687 (Bolt) 端口；redis 服务使用 7.x Alpine 轻量版本，同时充当 Celery 消息代理、限流计数器和 QA 缓存后端；postgres 服务使用 pgvector/pg16 镜像，集成了向量扩展的 PostgreSQL 16 数据库。

2.2 启动与验证

首次部署的完整步骤：第一步，克隆项目代码仓库并切换到目标版本标签；第二步，从 `.env.example` 复制环境配置文件并填写必要的 API Key 和访问密钥；第三步，执行 `docker-compose up -d` 命令启动所有服务，首次启动会自动拉取所需的 Docker 镜像；第四步，等待约 60 秒后执行 `docker-compose ps` 确认所有服务状态为 `healthy`；第五步，访问 `http://localhost:8000/health` 验证 API 服务是否正常响应；第六步，访问 `http://localhost:8000/docs` 查看自动生成的 OpenAPI 文档。

2.3 已知问题与解决方案

问题一：Docker Desktop for Windows 版本 v29.5.3 存在引擎响应超时的 Bug，表现为 `docker-compose up` 命令执行后长时间无响应，超时后报出 `context deadline exceeded` 错误。解决方案为卸载当前版本并降级安装 Docker Desktop v27.x，或者在 Windows 上改用 Podman Desktop 作为替代的容器运行时。

问题二：Milvus 依赖的 `etcd` 容器在首次启动时因缺少显式的 `listen-client-urls` 配置而 `CrashLoopBackoff`。解决方案已在 `docker-compose.yml` 中应用——为 `etcd` 服务添加了 `command` 指令，显式设置 `advertise-client-urls` 和 `listen-client-urls` 参数。

问题三：MinIO 在较新版本中强制要求设置 `root user` 和 `root password` 环境变量，旧版 `docker-compose` 配置仅设置了 `access key` 和 `secret key` 而遗漏了这两个必要变量，导致 MinIO 容器启动后立即退出。解决方案已在 `docker-compose.yml` 中补全 `MINIO_ROOT_USER` 和 `MINIO_ROOT_PASSWORD` 环境变量。

三、性能调优

3.1 Uvicorn Worker 配置

Uvicorn 的生产部署建议使用多 Worker 模式：对于纯 CPU 密集型负载（如嵌入推理），Worker 数不应超过 CPU 物理核心数；对于 IO 密集型负载（如 API 请求处理），Worker 数可以设置为 CPU 核心数的 2 至 4 倍。推荐的启动命令为：`uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 4 --loop uvloop --http httptools`。其中 `uvloop` 是 `libuv` 的 Python 绑定，相比默认的 `asyncio` 事件循环吞吐量提升约 20% 至 30%；`httptools` 是 C 语言实现的 HTTP 解析器，减少了解析开销。

3.2 LanceDB 索引优化

当向量数量超过 10,000 条时，建议为 LanceDB 创建 IVF_PQ 索引以替代默认的暴力搜索。索引参数建议为 `num_partitions=sqrt(N)`（其中 N 为向量总数）、`num_sub_vectors=32`（PQ 编码的子向量数， $512/32=16$ 维每个子向量）。索引构建操作较为耗时，建议在离线时段批量执行，或在新文档处理流水线中增量更新。

3.3 LLM 调用优化

减少 LLM Token 消耗的几项实用策略：第一，启用 QA 缓存——对于重复出现的高频问题，将 LLM 生成的答案缓存到 Redis 中，Key 为问题的 MD5 哈希与 `top_k` 参数的组合，TTL 设为 1 小时；第二，优化 Prompt 设计——将系统提示词从当前的约 200 个中文字符精简到约 80 个，通过减少 System Prompt 的长度降低每次调用的 Token 开销；第三，使用 LLM 熔断器——当连续 3 次调用失败时自动打开熔断器，后续请求在 30 秒内直接返回降级答案而不经 LLM，避免在故障期间持续消耗 API 配额。

四、备份与恢复

4.1 数据备份策略

生产环境需要定期备份以下四类数据：PostgreSQL 中的文档元数据和 QA 日志，建议使用 `pg_dump` 进行每日全量备份，保留最近 30 天的历史版本；LanceDB 中的向量数据，建议通过文件系统层面的目录复制进行每周全量备份（因为 LanceDB 的列式存储格式天然支持文件级别的快照）；Neo4j 中的知识图谱数据，建议使用 `neo4j-admin dump` 命令进行每日增量备份；MinIO 中的原始文档文件，建议配置 Bucket 级别的版本控制和跨区域复制。

4.2 灾难恢复流程

恢复流程按优先级排序：第一步，恢复 PostgreSQL 数据库——通过 `pg_restore` 将最近的备份文件还原到新的 PostgreSQL 实例，验证文档元数据的完整性（总数、时间范围）；第二步，恢复 MinIO 对象存储——将备份的 Bucket 数据同步到新的 MinIO 实例，使用 `mc mirror` 命令；第三步，恢复 Neo4j 图数据库——使用 `neo4j-admin load` 命令从备份文件重建图数据库；第四步，重建 LanceDB 向量索引——由于向量可以从源文档重新生成，LanceDB 数据的恢复优先级最低。如果原始文档完好，只需重新运行文档处理流水线即可重新生成所有向量。

五、安全配置

5.1 网络安全

生产环境必须限制各服务的网络暴露面。仅 API 服务的 8000 端口需要对用户网络可达；Milvus、Redis、PostgreSQL、Neo4j 和 MinIO 的内部端口不应直接暴露在公网上，仅允许 Docker 内部网络访问。如果必须从外部访问这些管理端口，应通过 SSH 隧道或 VPN 进行访问。

5.2 认证与授权

API 层应启用 API Key 认证中间件。设置环境变量 `RAG_API_KEY_ENABLED=true` 激活认证，设置 `RAG_API_KEY` 为不少于 32 个字符的随机字符串。API Key 支持两种传递方式：通过 `X-API-Key` 自定义请求头传递，或通过 `Authorization: Bearer` 标准头传递。健康检查端点 `/health` 和 API 文档端点 `/docs` 自动豁免认证检查。

敏感信息管理规则：LLM API Key、数据库密码、Redis 密码等必须通过环境变量或 Docker Secrets 注入，严禁硬编码在源代码或配置文件中提交到版本控制系统。所有密钥应定期轮换，最少每季度轮换一次。

六、监控指标参考

以下是生产环境正常运行时的各项关键指标的参考范围：QA 请求错误率应低于 1%（若超过 5% 持续 5 分钟触发告警）；QA 接口 P50 延迟应低于 2 秒，P99 延迟应低于 10 秒；LLM API 调用成功率应高于 98%；单次 QA 请求的平均 Token 消耗应控制在 1500 tokens 以下（Prompt + Completion 合计）；Celery Worker 的队列积压任务数不应超过 50；文档处理（含解析、分块、嵌入和图谱构建）的端到端延迟对于 10 页 PDF 应在 30 秒以内完成；向量库的可用磁盘空间应始终保留 20% 以上的余量；知识图谱的节点数和关系数应随文档上传量近似线性增长（异常偏离可能说明实体抽取出现了问题）。